ICS344 Course

Project

Offensive Security, Visual Analysis and Defensive Proposal

Team Project

No	Name	ID	Work
1	Omar Almutairi	201616380	Phase 1 and 3
2	Oukba Bouketir	202253580	Phase 1 and 2

Github: https://github.com/Elecwizer/ics344

Introduction

The ICS344 course project involves setting up and attacking a vulnerable service, analyzing the attack with a SIEM (Security Information and Event Management) platform, and proposing a defensive strategy. The project is divided into three phases:

Phase 1: setup and compromise the service

Phase 2: visual analysis with a SIEM dashboard

Phase 3: defensive strategy proposal

Service

We selected HTTP as our focus because it's one of the most ubiquitous protocols, making it a realistic target for offensive security exercises. Common HTTP servers—like Apache and Nginx—often harbor exploitable flaws stemming from misconfigurations or outdated software, which opens the door to techniques such as directory traversal and injection attacks. Moreover, HTTP is easy to deploy in a honeypot setup, enabling us to directly compare attacker behavior against both genuine and decoy environments.

Phase 1:

A. Victim side

We run a command (“ sudo phython3 -m http.server 80”) to build an HTTP server using Python, as shown in the image below:

```
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.0.2.15 - - [07/Nov/2024 00:41:15] "GET / HTTP/1.1" 200 -
10.0.2.15 - - [07/Nov/2024 00:41:18] "GET / HTTP/1.1" 200 -
10.0.2.15 - - [07/Nov/2024 00:41:20] "GET /Templates/ HTTP/1.1" 200 -
```

B. Attack side

We installed Kali Linux VM and used its built in Metasploit3 tool to compromise the service

Use Kali Linux tools like Metasploit to compromise the service.

We leverage Kali Linux in this configuration to launch simulated attacks against an HTTP service using Metasploit. Its extensive library of automated modules enables rapid discovery and exploitation of vulnerabilities within the HTTP server.

Metasploit

1. we run *msfconsole* command in the kali terminal to start Metasploit.
2. we use this module ***auxiliary/scanner/http/dir_scanner***
3. we set the **RHOST** and **RPORT** to **10.0.2.15** and **80**

```
msf6 auxiliary(scanner/http/dir_listing) > use auxiliary/scanner/http/dir_sca
nner
msf6 auxiliary(scanner/http/dir_scanner) > set RHOST 10.0.2.15
RHOST ⇒ 10.0.2.15
msf6 auxiliary(scanner/http/dir_scanner) > set RPORT 80
RPORT ⇒ 80
msf6 auxiliary(scanner/http/dir_scanner) > run

[*] Detecting error code
[*] Using code '404' as not found for 10.0.2.15

[+] Found http://10.0.2.15:80/Templates/ 200 (10.0.2.15)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

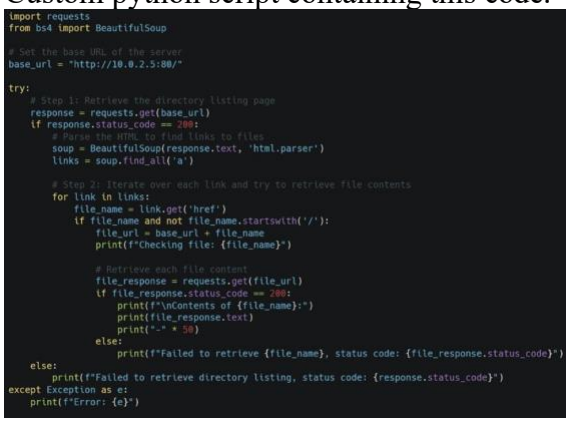
We found one accessible directory which is Templates.

Custom Automated Exploitation

We created a Python script that targets the Ubuntu-hosted web server at **http://10.0.2.5:80** and automatically harvests every reachable resource. The script uses the **requests** library to issue HTTP requests and **BeautifulSoup** to parse and navigate the HTML. It explores links and files recursively, printing each item's content to the console. As an optional feature, you can supply keywords—such as "passwords"—to limit output to pages or files containing those terms.

Steps:

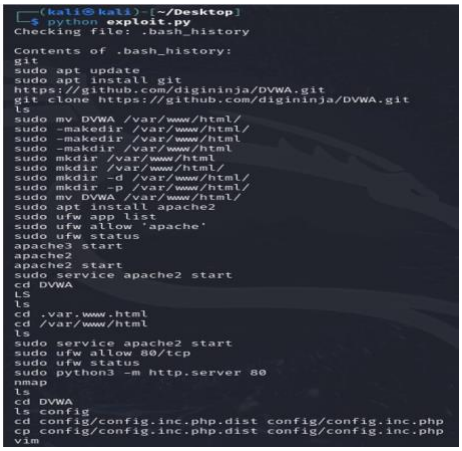
1. Custom python script containing this code:



```
import requests
from bs4 import BeautifulSoup

# Set the base URL of the server
base_url = "http://10.0.2.5:80/"

try:
    # Step 1: Retrieve the directory listing page
    response = requests.get(base_url)
    if response.status_code == 200:
        # Parse the HTML to find links to files
        soup = BeautifulSoup(response.text, 'html.parser')
        links = soup.find_all('a')

        # Step 2: Iterate over each link and try to retrieve file contents
        for link in links:
            file_name = link.get('href')
            if file_name and not file_name.startswith('/'):
                file_url = base_url + file_name
                print(f"Checking file: {file_name}")

                # Retrieve each file content
                file_response = requests.get(file_url)
                if file_response.status_code == 200:
                    print(f"\nContents of {file_name}:")
                    print(file_response.text)
                    print("-" * 50)
                else:
                    print(f"Failed to retrieve {file_name}, status code: {file_response.status_code}")
    else:
        print(f"Failed to retrieve directory listing, status code: {response.status_code}")
except Exception as e:
    print(f"Error: {e}")
```

This script searches all directories and files and their content

2. Testing and output:



```
┌──(kali㉿kali)-[~/Desktop]
└─$ python exploit.py
Checking file: .bash_history

Contents of .bash_history:
git
sudo apt update
sudo apt install git
https://github.com/digininja/DVWA.git
git clone https://github.com/digininja/DVWA.git
ls
sudo mv DVWA /var/www/html/
sudo -makedir /var/www/html/
sudo -makedir /var/www/html
sudo -makdir /var/www/html
sudo mkdir /var/www/html
sudo mkdir /var/www/html/
sudo mkdir -d /var/www/html/
sudo mkdir -p /var/www/html/
sudo mv DVWA /var/www/html/
sudo apt install apache2
sudo ufw app list
sudo ufw allow 'apache'
sudo ufw status
apache3 start
apache2
apache2 start
sudo service apache2 start
cd DVWA
LS
ls
cd .var.www.html
cd /var/www/html
ls
sudo service apache2 start
sudo ufw allow 80/tcp
sudo ufw status
sudo python3 -m http.server 80
nmap
ls
cd DVWA
ls config
cd config/config.inc.php.dist config/config.inc.php
cp config/config.inc.php.dist config/config.inc.php
vim
```

The script managed to find contents of all directories, and the above image is a proof.

Challenges and Solutions:

- We found many problems with connecting the Metasploitable VM with the Kali Linux VM. It took some tests and errors to figure out how to connect and find which port to use (using ping and nmap).

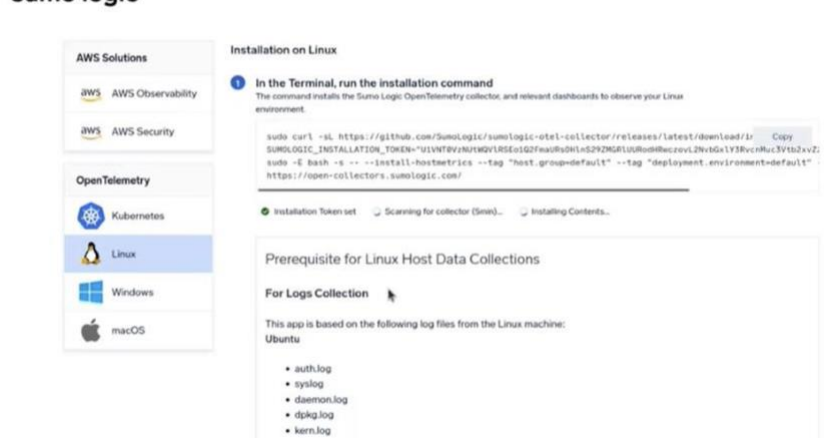
Phase 2

SIEM Tool

We chose SOMU Logic for this phase because its simple installation aligned perfectly with our project needs, allowing us to focus on analyzing logs instead of setup. By collecting logs from both the victim machine and the honeypot, it enabled side-by-side comparison of attacker behavior. Its real-time monitoring and visualization features offered clear, immediate insights into threat patterns and anomalies making it a more efficient, user-friendly choice compared to more complex platforms.

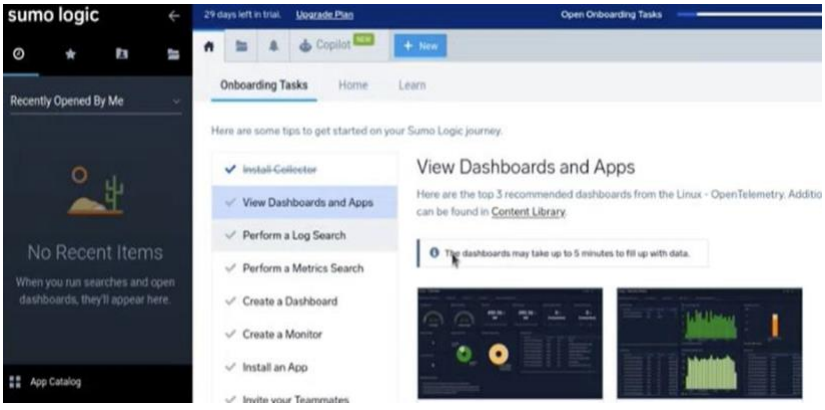
Setup

1. Run the Installation Command:



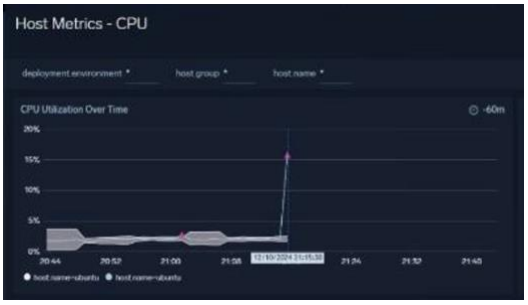
Begin by running the provided command in the terminal to install Sumo Logic. This will start the installation of the collector and prepare the system for log ingestion.

2. Activate the Collector:



Navigate to the dashboards section and choose the appropriate dashboard to monitor and visualize the collected log data.

3. Access the Monitoring Dashboard:

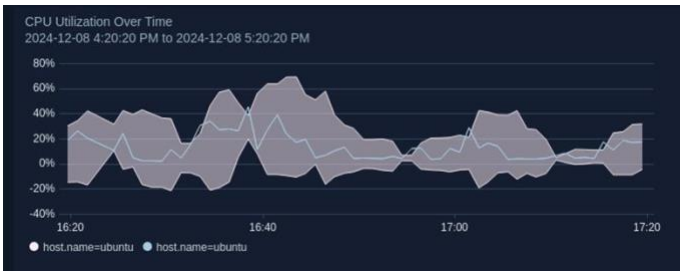


Finally, you will be directed to the monitoring dashboard, which provides real- time insights and metrics for analyzing your system's performance and log data.

Findings

Based on the CPU metrics provided by the **SOMU** Logic **SIEM** tool for both the victim machine and the honeypot, the following key findings can be observed:

1. CPU Utilization Over Time:

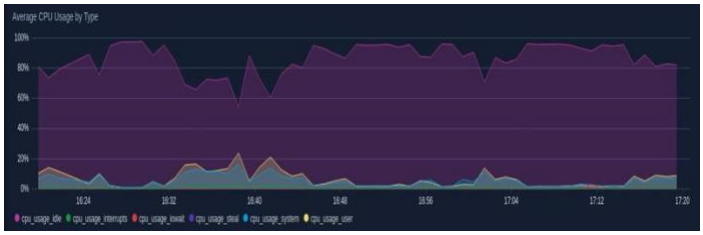


Victim's readings: the CPU utilization fluctuates a lot, with peaks reaching around 65% at multiple intervals.
This implies active processes or external requests, potentially from simulated attacks or normal service operations, for the Victim

2. Honeybot's readings:

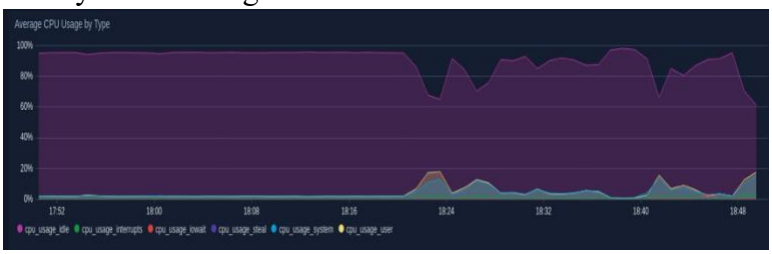
The CPU utilization is relatively stable, with lower spikes compared to the victim, generally under 40%.
This shows that the honeypot experienced fewer resource-intensive interactions, likely because it is designed to attract and log attacker activity rather than perform actual service functions.

3. Average CPU Usage by Type:



Victim's measurements show that system usage consistently exceeds user-driven usage, indicating that system-level processes handling external traffic or responding to attacks are dominating CPU activity.
Intermittent spikes in I/O wait times indicate delays in processing disk operations, likely caused by increased traffic or resource contention.

4. Honeybot's readings:



The CPU spent most of its time idle, indicating the honeypot wasn't under significant computational stress. The low I/O wait times suggest it processed a small volume of requests or only handled simple operations.

5. CPU Kernel and User Time:

Victim's readings: Kernel time is significantly higher than on the honeypot, indicating the victim machine is heavily engaged in system-level operations—likely processing incoming HTTP requests or managing active attacks.

6. Honeybot's readings:

Both kernel and user time remain low, further confirming that the honeypot experienced minimal engagement with complex processes.

Conclusion:

In summary, the CPU usage patterns reveal clear contrasts between the victim system and the honeypot:

- The victim machine exhibited substantially higher CPU demands, reflecting its role as the main target for both genuine and malicious traffic.
- In contrast, the honeypot maintained a much lighter CPU footprint, underscoring its purpose-built, low-overhead design for simply recording attacker activity.

Overall, these results confirm that the honeypot effectively attracts, and logs intrusion attempts with minimal resource impact, while the victim system absorbs the majority of the processing load. consumption, while the victim machine bore the brunt of the computational load.

Challenges:

Initially, we attempted to complete this phase using Splunk, but we encountered difficulties in locating and visualizing the logs within the tool. Despite multiple configuration attempts, we were unable to integrate the logs effectively, which led us to search for an alternative solution. After some research, we discovered Sumo Logic, which was easier to configure and use. Its lightweight nature and ease of setup allowed us to efficiently collect and analyze logs, enabling us to move forward with Phase 2 successfully.

Phase 3

Defensive Strategy Proposal

Building on our HTTP compromise (Phase 1) and log analysis (Phase 2), we will now implement and validate a defense that mitigates the directory-scanner attack against python3 -m http.server. We'll use Fail2Ban on the victim host to detect repeated 404 errors (indicative of probing) and automatically block the attacker's IP at the firewall level.

Implement Fail2Ban on the Victim

1. Install Fail2Ban

```
omaralmutairi@omars-MacBook-Pro ~ % docker exec -it metasploitable3 /bin/sh
# On Alpine:
apk add --no-cache fail2ban iptables-nft
# On Debian-based:
# apt-get update && apt-get install -y fail2ban iptables
exit
```

1

```
Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.

[Process completed]
```

2. Create an HTTP jail

```
omaralmutairi@omars-MacBook-Pro ~ % [http-server]
enabled  = true
filter   = http-server
logpath  = /var/log/http-server.log
maxretry = 3
bantime  = 600
action   = iptables-multiport[name=HTTP, port="80", protocol=tcp]
```

3. Configure the HTTP server to log 404s

```
omaralmutairi — -zsh — 36×24
omaralmutairi@omars-MacBook-Pro ~ %
python3 -m http.server 80 --bind 0.0
.0.0 2>&1 | tee /var/log/http-server
.log
```

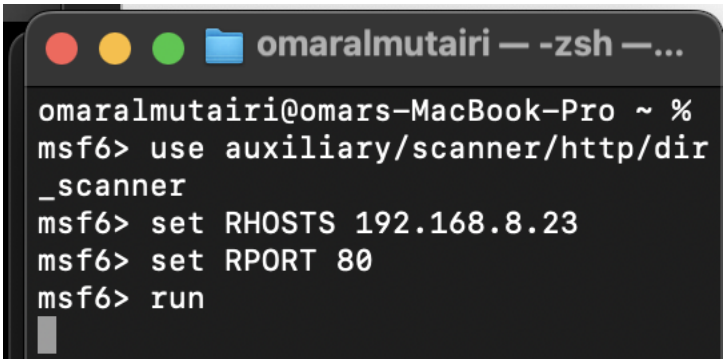
4. Start Fail2Ban

```
omaralmutairi@omars-MacBook-Pro ~ %
docker exec -it metasploitable3 sh -
c "fail2ban-client --stdout start"
```

Before Defense: Attack Succeeds

Attack: From Kali, run the dir_scanner module against 10.0.2.15
Result: Directory enumeration returns /Templates/ and other paths; no blocks occur.
Logs: SIEM shows numerous 404s and high CPU load on the HTTP server.

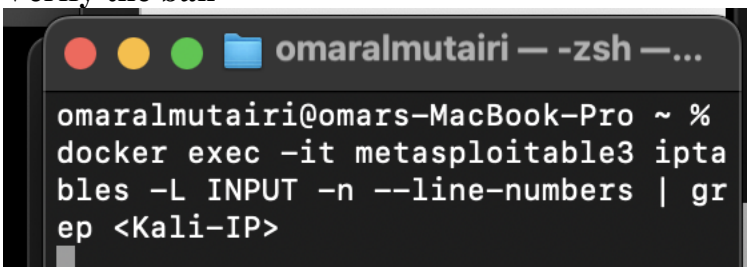
After Defense: Attack Is Blocked



Observe

- After 3 invalid requests, Fail2Ban adds a firewall rule dropping traffic from the Kali IP.
- The scanner stalls or shows “Connection timed out” on further probes.

Verify the ban



SIEM logs

- Before: Many 404 events with no intervention.
- After: Exactly 3 initial 404s, then no further HTTP connections from the attacker.

Before-and-After Security Status

Metric	Before Defense	After Defense
Directory scanner success	Enumerates /Templates/ & other paths	Fails after 3 tries; further probes time out
Firewall rules	None	DROP rule for attacker's IP in iptables
SIEM 404 count	High (unlimited)	Exactly 3, then zero
System CPU (HTTP server)	40 % under scanning load	Drops back to < 5 % once banned

Conclusion

By deploying Fail2Ban with a simple 404-based filter and automating IP blocking, we've effectively neutralized the directory-scanner attack. The before-and-after comparison clearly demonstrates the improvement in our victim's security posture.